

FootballClub — priprema za obranu projekta

ASP.NET Core seminar · rubrika v2 (ukupno 70 bodova + 3 extra) · stanje: 10. srpnja 2026.

Aplikacija: sustav za upravljanje nogometnim klubom (Dinamo Zagreb) — igrači, osoblje, treninzi, utakmice, transferi, statistike, liga. ASP.NET Core 8 MVC + REST API, EF Core + SQL Server, Identity + JWT + Google OAuth, Gemini AI, MCP server, 109 automatskih testova, deploy na Azure.

1. Pregled po kriterijima ocjenjivanja

Kriterij	Bodovi	Status	Ključna lokacija u kodu
Deploy na cloud/VM	3	DA — Azure	.github/workflows/master_footballclub-web.yml
Testovi za sve API endpointe	2	DA — 109 testova	FootballClub.Tests/Api/* (13 datoteka)
Playwright scenarij 10 koraka	+3	DA — bonus	FootballClub.Tests/E2E/PlaywrightScenarioTests.cs
AI integracija (unos podataka)	3	DA — Gemini	Web/Features/Ai/*
Global search (izbornici + stranice + podaci)	2	DA	wwwroot/js/global-search.js
Logging mehanizam (file ili API)	2	DA — API	Web/Features/ActivityLog/*
Responsive mobile/web UI	2	DA	wwwroot/css/site.css
CRUD bez grešaka	2	DA	Web/Features/<Feature>/*ApiController.cs
Expose MCP + agentic IDE	2	DA — /mcp	Web/Features/<Feature>/*McpTools.cs
Dojam funkcionalnosti (bez crasheva)	12	obrambeni mehanizmi ugrađeni	—
Usmeno ispitivanje	40	ovaj dokument	—

Napomena: tablica u zaglavlju bodova prati novu verziju kriterija (Kriteriji-projekt-pragovi.pdf). Svi tehnički kriteriji su implementirani; Playwright scenarij nosi 3 dodatna boda koja mogu kompenzirati gubitke drugdje.

2. Arhitektura — velika slika (reci ovo prvo ako te pitaju „opiši projekt“)

Jedan web projekt FootballClub organiziran po **feature folderima**: sve za jednu funkcionalnost (MVC kontroler, API kontroler, DTO-i, view-modeli, servisi, MCP alati) živi zajedno u Web/Features/<Feature>/ — Players, Staff, Training, Matches, Clubs, Stadiums, Transfers, League, Auth, ActivityLog, Ai... Zajednički tipovi su u Web/Common (ApiControllerBase, PagedResultDto). Domenski sloj je odvojen: Models/ (entiteti + Mapping ekstenzije), Data/ (DbContext, DataSeeder), Repositories/ (repozitoriji nad DbContextom).

Entiteti: Club, Player, Staff (tablica „Coaches“), Match, Stadium, PlayerStat (N:N igrač-utakmica), Transfer, TrainingSession, PlayerScheduleItem, LeagueStanding, ActivityLog + Identity (AppUser, uloge s int ključem). UI je „SPA-style“: Razor stranice su ljuska, podaci se dohvaćaju fetchom s REST API-ja uz JWT.

3. Deploy na Azure (3 boda)

Što: Azure App Service (free tier F1, app „footballclub-web“) + Azure SQL serverless, CI/CD kroz GitHub Actions.

- Workflow .github/workflows/master_footballclub-web.yml: na push na master — job „build“ (dotnet build + publish, upload artifakta) pa job „deploy“ (azure/webapps-deploy@v3 s publish profilom iz GitHub Secrets).
- Tajne (connection string, JWT ključ, Google OAuth, Gemini ključ) NISU u repozitoriju — appsettings.json ima prazne vrijednosti, prave dolaze iz App Service konfiguracije (environment varijable).

- Azure SQL serverless se auto-pauzira i budi 30–60 s: zato `EnableRetryOnFailure(10, 30s)` u `Web/Program.cs` — bez toga bi prvi request nakon pauze pao.
- Migracije se primjenjuju automatski na startu (`MigrateAsync()`); seeding je best-effort u try/catch da pad seeda ne sruši aplikaciju.

4. Testovi — 109 komada (2 boda + 3 extra za Playwright)

API testovi (98): 13 datoteka u `FootballClub.Tests/Api/`, po jedna za svaki API kontroler. Uzorak po endpointu: happy path (200/201), not found (404), nevaljan payload (400), autorizacija (401/403).

- `TestWebApplicationFactory` diže CIJELU pravu aplikaciju in-memory (`WebApplicationFactory<Program>`) — zato `Program.cs` završava s `public partial class Program { }`. Testovi idu kroz pravi middleware pipeline.
- Zamjene za test: SQL Server → EF InMemory (jedinstveno ime baze po testu = izolacija), Gemini → `FakeAiClient` (deterministički), antiforgery → `NoOpAntiforgery`.
- `TestJwtTokenGenerator` izdaje pravi JWT s ulogom (User/Admin) pa se testira i autorizacija.
- **MCP testovi (10):** `Tests/Mcp/McpToolsTests.cs` zove alate direktno nad seedanom bazom; `McpServerIntegrationTests.cs` spaja pravi MCP klijent kroz Streamable HTTP transport.

Playwright scenarij 10 koraka (+3 boda)

`FootballClub.Tests/E2E/PlaywrightScenarioTests.cs` — jedan test `TenStepUserJourney` u headless Chromiumu, svaki korak označen komentarom:

1. Anonimni posjet `/team-roster` → auth gate preusmjeri na `/Account/Login`
2. Registracija novog računa kroz UI formu
3. Slijetanje na Dashboard — topbar pokazuje korisnika i ulogu
4. Navigacija na Players kroz sidebar izbornik
5. CREATE: popunjavanje forme `/Player/Create` (Zvonimir Playwright, #42, Dinamo)
6. READ: igrač se pojavljuje u roster tablici
7. GLOBAL SEARCH: Ctrl+K → pretraga → klik rezultata vodi na profil igrača
8. UPDATE: promjena broja dresa 42 → 99, provjera u tablici
9. DELETE: soft delete s potvrdom `confirm()` dijaloga → red pokazuje „Deleted“
10. Odjava → auth gate ponovno blokira aplikaciju

Infrastruktura (odlična priča za usmeno): `E2WebApplicationFactory` gradi aplikaciju DVAPUT iz istog buildera — jednom za `TestServer` (koji `WebApplicationFactory` zahtijeva) i jednom na pravom Kestrelu s random portom (browser ne može do in-memory `TestServera`). Svaki host dobiva VLASTITU in-memory bazu jer dva `Program` replaya seedaju paralelno — dijeljena baza završi s duplikatima. Chromium se automatski instalira pri prvom dotnet test.

5. AI integracija — Gemini (3 boda)

Dvije funkcionalnosti: (a) unos podataka prirodnim jezikom koji pre-fila Create forme za igrača/osoblje/trening, (b) chatbot asistent na dashboardu. Provider: Google Gemini 2.5 Flash (besplatni tier).

Tok unosa podataka (nauči ovaj slijed!)

1. Korisnik u panel (`_AiEntryPanel.cshtml`) upiše npr. „23-godišnji hrvatski veznjak Luka M., dres 10, vrijedi 50M, ugovor do 2027, igra za Dinamo“ i klikne Generate.
2. JS POST-a na `AiController (/ai/extract/player | staff | training; [Authorize] + antiforgery)`.
3. Kontroler gradi `AiEntitySchema` — polja s tipovima + instrukcije koje uključuju STVARNE podatke iz baze (imena klubova, enum pozicija, imena trenera) da model bira postojeće vrijednosti.
4. `GeminiAiClient.ExtractAsync` zove `generateContent` sa **structured outputom** (`responseMimeType: application/json + responseSchema, temperatura 0`) — model MORA vratiti validan JSON točno tih polja.

5. Odgovor se tretira kao NEPOUZDAN: defanzivno parsiranje (Str/Int/Dec/Bool/IsoDate), mapiranje imena u ID-jeve (ResolveName: exact pa fuzzy match, višeznačnost → upozorenje), validacija enuma (MapEnum).
6. JS popuni formu — korisnik pregleda i SAM sprema kroz normalni Create POST. **Human-in-the-loop: AI nikad ne piše direktno u bazu.**

Chatbot

AssistantApiController (POST /api/assistant/chat, [Authorize] — bez logina nema asistenta).

BuildClubContext() složi tekstualni snapshot kluba iz baze (igrači, osoblje, tablica, utakmice, treninzi) i ubaci ga u system prompt — tzv. **context stuffing / grounding**. Limit 12 zadnjih poruka.

Arhitektura: sučelje IAIClient ima dvije implementacije — GeminiApiClient kad je Ai:ApiKey konfiguriran, inače NullApiClient koji uredno vraća „AI nije konfiguriran“. Aplikacija i testovi rade i bez ključa (Program.cs bira pri registraciji u DI).

6. Global search (2 boda)

Tražilica u topbaru (Ctrl+K), wwwroot/js/global-search.js + markup Views/Shared/_GlobalSearch.cshtml.

Hibridni pristup — pokriva sve tri stavke nove rubrike:

- **Izbornici i stranice:** statični indeks PAGES s hrvatskim i engleskim ključnim riječima; normalize() skida dijakritike (NFD dekompozicija + ručno đ→d) pa „igraci“ nađe „Igrači“. Instant, bez servera.
- **Podaci:** paralelni fetch na /api/players/search, /api/staff/search, /api/training/search — svi vraćaju uniformni oblik {id, text, subtitle, url}.
- **Tehnički detalji:** debounce 200 ms; AbortController + sekvencijski broj (stariji odgovor ne smije pregaziti noviji); tipkovnička navigacija (strelice/Enter/Escape); mousedown umjesto click da panel ne nestane prije navigacije; Activity Log stavka vidljiva samo adminu (adminOnly + Auth.getRole()).

7. Logging mehanizam — audit log u bazi + API (2 boda)

Tko je što napravio, kada, s koje IP adrese, uspješno ili ne. Tri sloja:

- **Automatski:** ActivityLogActionFilter registriran GLOBALNO (Program.cs) hvata svaki POST/PUT/PATCH/DELETE — i MVC form postove i JSON API pozive — bez ijedne linije po kontroleru.
- **Eksplícitno:** auth događaji (login, neuspjeli login, registracija) logiraju se ručno s više konteksta u AuthController; kontroler nosi [SkipActivityLog] da se ne dupla s filterom.
- **Sink:** DbActivityLogger (iza sučelja IActivityLogger) piše u tablicu ActivityLogs. Dvije defenzivne odluke: VLASTITI DI scope (poseban DbContext — audit ne sudjeluje u transakciji requesta) i PROGUTANE iznimke (pad audita ne smije srušiti operaciju koja se auditira).
- **Pregled:** ActivityLogsApiController — [Authorize(Roles="Admin")], paginacija + filtri po korisniku/akciji; MVC stranica /ActivityLog je ljuška koja fetcha s bearer tokenom. Time je zadovoljena „file ili API“ varijanta rubrike — naša je API.

8. Responsive UI (2 boda)

- Media queryji u wwwroot/css/site.css; sidebar se na mobitelu pretvara u drawer s hamburger gumbom i overlay backdropom (navToggle + sidebarOverlay u _Layout.cshtml, logika u site.js).
- Tablice se na mobitelu pretvaraju u KARTICE — nema horizontalnog scrollanja (tr postaje blok s labelama).
- Statistike na dashboardu idu u 2×2 grid na užim ekranima; meta viewport u layoutu.
- Demo na obrani: suzi prozor ili DevTools responsive mode.

9. CRUD bez grešaka (2 boda)

Puni CRUD kroz REST API + MVC stranice za sve entitete. Ogledni primjer za obranu:

Web/Features/Players/PlayersApiController.cs:

- GET /api/players — pretraga + paginacija (PagedResultDto iz ApiControllerBase, Range validacija parametara).
- POST/PUT — provjera postojanja FK-ova (klub, trening) → 404 s porukom; route id vs. payload id → 400; [ApiController] automatski vraća 400 za nevaljan model (DataAnnotations na DTO-ima).
- DELETE — SOFT DELETE (IsDeleted = true), samo Admin; sva čitanja filtriraju !IsDeleted.
- Autorizacija po metodi: čitanje anonimno, pisanje [Authorize(Roles="Admin, User")], brisanje samo Admin.
- EF detalji: DeleteBehavior.Restrict za Match/Transfer prema klubovima (brisanje kluba ne kaskadira kroz utakmice) u OnModelCreating; DataSeeder je idempotentan (guard: if (!context.Players.Any())).

10. MCP server + agentic IDE (2 boda)

Što je MCP: Model Context Protocol — otvoreni protokol kojim AI agenti (Claude Code, VS Code Copilot, Cursor) otkrivaju i pozivaju alate koje server izlaže. Naša aplikacija je MCP SERVER na ruti /mcp (paket ModelContextProtocol.AspNetCore 1.4.0, službeni C# SDK; Streamable HTTP transport).

- Registracija u Program.cs: `AddMcpServer().WithHttpTransport().WithToolsFromAssembly() + app.MapMcp("/mcp")`.
- 14 alata u 8 feature foldera, svaki <Feature>McpTools.cs pored svog API kontrolera: `list_clubs`, `get_club`, `list_players`, `get_player` (jedini write!), `get_player_stats`, `list_staff`, `get_staff`, `list_matches`, `get_match`, `list_stadiums`, `list_training_sessions`, `list_transfers`, `get_league_standings`.
- Statične klase s [McpServerToolType]/[McpServerTool]/[Description]; parametar `ApplicationDbContext` se razrješava iz DI po pozivu (ne ulazi u JSON schemu alata). Read alati zrcale anonimne GET endpointe i koriste `ToDto()` mapping ekstenzije; `create_player` kopira validaciju iz `PlayersApiController`.
- **Zašto bez autentikacije:** ista razina povjerenja kao postojeći anonimni GET-ovi; lokalna dev/demo površina, ne javni API. Ako pitaju za sigurnost: `AddAuthorizationFilters() + .RequireAuthorization()` na `MapMcp` bi to zaključalo — svjesno izostavljeno radi jednostavnosti demoa.
- **Demo:** pokreni app (`dotnet run`, `http://localhost:5203`) pa `claude mcp add --transport http footballclub http://localhost:5203/mcp` — zatim u IDE-u: „prikaži ljestvicu HNL-a” ili „dodaj igrača...”.

11. Dojam funkcionalnosti — ugrađeni obrambeni mehanizmi (12 bodova)

- Hladan start Azure SQL-a pokriven retryjem (10 pokušaja / 30 s).
- Seed u try/catch — pad seedanja ne ruši aplikaciju.
- AI bez ključa degradira u NullAiClient umjesto pada; Gemini greške vraćaju uredan JSON error.
- Audit log guta vlastite iznimke; global search catcha neuspjele fetcheve i vrati prazno.
- Soft delete čuva referencijalni integritet (nema orphan FK-ova).

12. Usmeno ispitivanje — vjerojatna pitanja i odgovori (40 bodova!)

P: Objasni middleware pipeline i zašto je redosljed bitan.

U Program.cs: `UseStaticFiles` → `UseHttpsRedirection` → `UseRequestLocalization` → `UseRouting` →

UseAuthentication PRIJE UseAuthorization (prvo utvrdi TKO si iz tokena, onda ŠTO smiješ) →

`MapRazorPages/MapControllers/MapMcp`. Statika je prva da css/js ne prolaze kroz auth.

P: Kako radi prijava? Objasni cijeli tok.

POST /api/auth/login → `UserService.AuthenticateAsync` provjeri lozinku kroz Identity

(`UserManager.CheckPasswordAsync`) → `CreateToken` gradi JWT (HMAC-SHA256; claims: id, ime, email, ULOGA; traje 120 min). Browser sprema token u `localStorage` (auth.js) i šalje ga kao `Authorization: Bearer` header. Server validira issuer, audience, potpis i istek (`ClockSkew=Zero`) u `JwtBearer` konfiguraciji.

P: Gdje je stvarna sigurnost ako je redirect na login klijentski?

Dvoslojno: klijentski redirect (inline skripta u _Layout) je samo UX. Prava zaštita je [Authorize] na API kontrolerima — podaci se ionako dohvaćaju fetchom s tokenom, pa i da netko zaobiđe redirect, API vraća 401.

P: Kako radi Google prijava?

Challenge → Google → callback čita claimove iz privremenog External COOKIEJA, nađe/stvori lokalnog usera (UserService.AuthenticateExternalAsync), izda NAŠ standardni JWT, pa bridge stranica /Account/ExternalLogin spremi token u localStorage. Poanta: vanjski identitet se pretvara u naš token, ostatak aplikacije ne zna razliku.

P: Koje uloge postoje i što smiju?

User — sve funkcionalnosti + chatbot. Admin — dodatno upravljanje računima (/Accounts, /api/users: promocija u admina, deaktivacija) i audit log. Registracijom se dobiva uloga User; uloge su u JWT claimu pa [Authorize(Roles=...)] radi bez dodatnog upita u bazu.

P: Zašto je DbActivityLogger singleton, a DbContext scoped?

DbContext nije thread-safe i živi po requestu (scoped). Logger je singleton jer ne drži DbContext direktno — drži IServiceScopeFactory i za svaki zapis stvara vlastiti kratkoživi scope. Bonus: audit zapis time ne sudjeluje u transakciji requesta i ne bude rollbackan s njom.

P: Objasni ApplicationDbContext.

Nasljeđuje IdentityDbContext<AppUser, IdentityRole<int>, int> — Identity tablice i domenske u istoj bazi, int ključevi. OnModelCreating konfigurira veze: Match ima dva FK-a na Club (HomeClub/AwayClub) s DeleteBehavior.Restrict; PlayerStat je N:N veza igrač-utakmica; Transfer Restrict na klubove, Cascade na igrače.

P: Zašto soft delete umjesto pravog brisanja?

Player/Staff/TrainingSession imaju povijesne veze (statistike, transferi, raspored) — pravo brisanje bi ili kaskadno obrisalo povijest ili ostavilo orphan FK-ove. IsDeleted=true čuva podatke; admin ih vidi i može Restore.

P: Kako testovi dižu aplikaciju bez pravog servera i baze?

WebApplicationFactory<Program> pokreće cijeli Program.cs in-memory (TestServer). U ConfigureWebHost se zamijene: SQL Server → EF InMemory (ime baze = GUID po testu → izolacija), Gemini → FakeAiClient, antiforgery → no-op. HttpClient iz factoryja gađa prave rute kroz pravi pipeline.

P: Po čemu je Playwright test poseban? (ispričaj bug!)

Browser ne može do TestServera, pa E2EWebApplicationFactory gradi app DVAPUT — TestServer + pravi Kestrel na random portu. Bug koji sam našao: oba builda ponovno izvrše Program.cs (svaki na svojoj niti) uključujući seedanje — dijeljena in-memory baza završi s DUPLIKATIMA (registracija 500: „Sequence contains more than one element“). Fix: svaki host dobije vlastito ime baze. Druga zamka: ime se mora izračunati IZVAN AddDbContext options lambdae, jer je DbContextOptions scoped pa se lambda izvršava po requestu — brojač unutra bi svakom requestu dao praznu bazu.

P: Kako AI unos garantira valjan JSON?

Gemini structured output: responseSchema definira točna polja i tipove, responseMimeType=application/json, temperatura 0. Uz to server svejedno defanzivno parsira svaki element (AI output je nepouzdan input) i mapira imena u ID-jeve uz upozorenja kod višeznačnosti. Korisnik uvijek pregleda prije spremanja (human-in-the-loop).

P: Zašto se repozitoriji zovu MockRepository ako rade s bazom?

Povijesno ime iz ranih labova kad su vraćali hardkodirane podatke; kasnije su prespojeni na DbContext, ime je ostalo. Iskren odgovor + pokazuje da razumiješ evoluciju koda. (PlayerMockRepository.GetAll() = _context.Players.Where(!IsDeleted).ToList()).

P: Što je MCP i po čemu se razlikuje od REST API-ja?

REST je za ljude/frontend — fiksne rute, klijent mora znati unaprijed što zvati. MCP je za AI agente — agent na runtime OTKRIJE popis alata s opisima i JSON schemama parametara pa sam odluči koje pozvati. Isti domenski sloj, druga „vrata“: naši MCP alati zrcale API kontrolere i dijele mapping ekstenzije.

P: Kako je riješena lokalizacija?

RequestLocalization s podržanim kulturama hr (default) i en-US — utječe na formate datuma/brojeva (npr. custom datepicker čita data-culture i parsira dd.MM.yyyy odnosno M/d/yyyy).

13. Checklist na dan obrane

- `dotnet test` → mora biti 109/109 zeleno (uključuje Playwright E2E; prvi put skida Chromium).
- Probudi Azure prije termina: otvori produkcijski URL i pričekaj prvi (spori) request — baza se budi 30–60 s.
- Provjeri da AI radi na produkciji (chatbot + AI unos) — ključ mora biti u App Service konfiguraciji.
- MCP demo lokalno: `dotnet run` pa `claude mcp add --transport http footballclub http://localhost:5203/mcp` — probaj „prikaži ljestvicu“ prije obrane.
- Ako želiš Playwright pokazati uživo s vidljivim browserom: u `PlaywrightScenarioTests.cs` stavi `Headless=false` (+ po želji `SlowMo`).
- Prođi mobilni prikaz (drawer, kartice umjesto tablica) i `Ctrl+K` pretragu.
- Pripremi login podatke (registriraj demo račun unaprijed ili koristi seedanog admina).
- Otvori ovaj PDF i `Program.cs` — većina usmenih pitanja kreće od njega.

Generirano 10. 7. 2026. iz stvarnog stanja koda (git grana master + nova E2E/MCP implementacija). Brojevi testova i putanje datoteka odgovaraju trenutnom repozitoriju.